

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Assignment-Slot-B (Answer)

Programme : B.E / B.Tech (CSE - AI)
Course Code : CSA1707
Course Name : ARTIFICIAL INTELLIGENCE
Max Marks : 100
Student Name : J O Joanna Divine
Register No. : 192572003

Question 1

Develop a small intelligent student academic advisory agent using First-Order Logic (FOL). The system should include at least 8 facts, at least 6 rules, at least 2 variables, at least 2 quantified statements, and at least 2 queries. Demonstrate the working of the system using unification, forward chaining, backward chaining, and resolution. Finally, explain how the FOL-based approach differs from an equivalent propositional-logic implementation.

Answer

1. Problem Setup: Student Academic Advisory Agent

The agent advises students on which courses they are eligible to take, based on the courses they have already completed, the prerequisites of a course, and their GPA. The agent is built as a First-Order Logic (FOL) knowledge base consisting of a set of ground facts and a set of general rules (Horn clauses) expressed using predicates, constants, variables, and quantifiers.

Predicates used:

- Student(x) - x is a registered student
- Course(c) - c is a course offered by the department
- Enrolled(x, c) - student x is currently enrolled in course c
- Completed(x, c) - student x has completed course c
- Prerequisite(c, p) - course p is a prerequisite of course c
- Instructor(i, c) - i is the instructor handling course c
- GPA(x, g) - student x has grade-point average g
- Greater(a, b) - numeric value a is greater than b
- CanEnroll(x, c) - student x is eligible to enroll in course c
- HonorRoll(x) - student x qualifies for the honor roll
- GoodStanding(x, c) - student x is in good academic standing for course c
- NeedsAdvising(x, c) - student x should meet an advisor about course c

2. Knowledge Base - Facts (10 ground facts)

F1: Student(Joanna)
F2: Student(Ravi)
F3: Course(AI)

F4: Course(DataStructures)
F5: Prerequisite(AI, DataStructures)
F6: Completed(Joanna, DataStructures)
F7: Completed(Ravi, DataStructures)
F8: Enrolled(Joanna, AI)
F9: Instructor(DrKumar, AI)
F10: GPA(Joanna, 8.5)

3. Knowledge Base - Rules (6 rules, with variables and quantifiers)

Variables x , c , p , i and g range respectively over students, courses, prerequisite courses, instructors and GPA values. Each rule below is universally quantified ($\forall$), so the rules themselves are the required ≥ 2 quantified statements (R1 and R2 are shown fully quantified; the same convention applies to R3-R6).

R1: $\forall x \forall c \forall p$ [Course(c) $\wedge$ Prerequisite(c, p) $\wedge$ Completed(x, p)] $\rightarrow$ CanEnroll(x, c)
R2: $\forall x \forall g$ [GPA(x, g) $\wedge$ Greater($g, 8.0$)] $\rightarrow$ HonorRoll(x)
R3: $\forall x \forall c$ [Enrolled(x, c) $\wedge$ CanEnroll(x, c)] $\rightarrow$ GoodStanding(x, c)
R4: $\forall x \forall c \forall i$ [Instructor(i, c) $\wedge$ Enrolled(x, c)] $\rightarrow$ HasInstructor(x, i)
R5: $\forall x$ HonorRoll(x) $\rightarrow$ RecommendAdvancedCourse(x)
R6: $\forall x \forall c$ [CanEnroll(x, c) $\wedge$ $\neg$ Enrolled(x, c)] $\rightarrow$ NeedsAdvising(x, c)

Variables: x and c appear in every rule (≥ 2 variables satisfied); p , g and i appear as additional variables in R1, R2 and R4. Quantified statements: every rule (R1-R6) is a universally quantified implication, so ≥ 2 quantified statements is satisfied.

4. Queries (2 queries)

Q1: CanEnroll(Joanna, AI) ? - "Is Joanna eligible to enroll in AI?"
Q2: HonorRoll(Joanna) ? - "Does Joanna qualify for the honor roll?"

5. Demonstrating Unification

To apply rule R1 to fact F5 and F6, the variables of R1 must be unified with the constants in the facts.

R1 body term: $\text{Prerequisite}(c, p)$ unified with F5: $\text{Prerequisite}(\text{AI}, \text{DataStructures})$

Substitution $\theta_1 = \{c/\text{AI}, p/\text{DataStructures}\}$

R1 body term: $\text{Completed}(x, p)$ unified with F6: $\text{Completed}(\text{Joanna}, \text{DataStructures})$, using θ_1 ($p/\text{DataStructures}$ already fixed)

Substitution $\theta_2 = \{x/\text{Joanna}\}$

Combined most general unifier (MGU): $\theta = \{x/\text{Joanna}, c/\text{AI}, p/\text{DataStructures}\}$. Applying θ to the head of R1, $\text{CanEnroll}(x, c)$, yields the new fact $\text{CanEnroll}(\text{Joanna}, \text{AI})$.

6. Forward Chaining (data-driven derivation)

Forward chaining starts from the known facts and repeatedly fires rules whose premises are already satisfied, adding new facts to the knowledge base until no more rules apply.

- Step 1: $F_3 (\text{Course}(\text{AI})) \wedge F_5 (\text{Prerequisite}(\text{AI}, \text{DataStructures})) \wedge F_6 (\text{Completed}(\text{Joanna}, \text{DataStructures}))$ satisfy the body of R1 with $\theta = \{x/\text{Joanna}, c/\text{AI}, p/\text{DataStructures}\} \rightarrow$ derive $\text{CanEnroll}(\text{Joanna}, \text{AI})$.
- Step 2: $F_8 (\text{Enrolled}(\text{Joanna}, \text{AI})) \wedge \text{CanEnroll}(\text{Joanna}, \text{AI})$ [from Step 1] satisfy R3 $\rightarrow$ derive $\text{GoodStanding}(\text{Joanna}, \text{AI})$.
- Step 3: $F_9 (\text{Instructor}(\text{DrKumar}, \text{AI})) \wedge F_8 (\text{Enrolled}(\text{Joanna}, \text{AI}))$ satisfy R4 $\rightarrow$ derive $\text{HasInstructor}(\text{Joanna}, \text{DrKumar})$.
- Step 4: $F_{10} (\text{GPA}(\text{Joanna}, 8.5)) \wedge \text{Greater}(8.5, 8.0)$ satisfy R2 $\rightarrow$ derive $\text{HonorRoll}(\text{Joanna})$.
- Step 5: $\text{HonorRoll}(\text{Joanna})$ [Step 4] satisfies R5 $\rightarrow$ derive $\text{RecommendAdvancedCourse}(\text{Joanna})$.

After Step 1 and Step 4, both queries Q1 ($\text{CanEnroll}(\text{Joanna}, \text{AI})$) and Q2 ($\text{HonorRoll}(\text{Joanna})$) are already present in the knowledge base, so forward chaining answers “yes” to both.

7. Backward Chaining (goal-driven derivation)

Backward chaining starts from the query (goal) and works backwards, trying to satisfy the premises of a rule whose head matches the goal.

- Goal: $\text{CanEnroll}(\text{Joanna}, \text{AI})$?
- Match goal with head of R1: $\text{CanEnroll}(x, c)$ with $\theta = \{x/\text{Joanna}, c/\text{AI}\}$.
- New sub-goals: $\text{Course}(\text{AI})?$, $\text{Prerequisite}(\text{AI}, p)?$, $\text{Completed}(\text{Joanna}, p)?$
- $\text{Course}(\text{AI})$ matches fact F3 $\rightarrow$ satisfied.
- $\text{Prerequisite}(\text{AI}, p)$ matches fact F5 with $p = \text{DataStructures}$ $\rightarrow$ satisfied, p bound to DataStructures .
- $\text{Completed}(\text{Joanna}, \text{DataStructures})$ matches fact F6 $\rightarrow$ satisfied.
- All sub-goals proved $\rightarrow$ original goal $\text{CanEnroll}(\text{Joanna}, \text{AI})$ is TRUE.

Similarly, for Goal: $\text{HonorRoll}(\text{Joanna})$? — R2 is matched ($\theta = \{x/\text{Joanna}\}$), giving sub-goals $\text{GPA}(\text{Joanna}, g)?$ and $\text{Greater}(g, 8.0)?$. $\text{GPA}(\text{Joanna}, 8.5)$ matches F10 ($g = 8.5$), and $\text{Greater}(8.5, 8.0)$ is true, so both sub-goals succeed and $\text{HonorRoll}(\text{Joanna})$ is proved TRUE.

8. Demonstrating Resolution

To use resolution, rule R1 and the relevant facts are first converted to clausal form (disjunction of literals), and the query is negated and added to the clause set. Resolution then derives the empty clause, proving the query.

Clause form of R1: $\neg \text{Course}(c) \vee \neg \text{Prerequisite}(c,p) \vee \neg \text{Completed}(x,p) \vee \text{CanEnroll}(x,c)$

Fact clauses: $\text{Course}(\text{AI}). \text{Prerequisite}(\text{AI}, \text{DataStructures}).$
 $\text{Completed}(\text{Joanna}, \text{DataStructures}).$

Negated goal: $\neg \text{CanEnroll}(\text{Joanna}, \text{AI})$

Resolve R1-clause with $\text{Course}(\text{AI})$ [$\theta = \{c/\text{AI}\}$]:

$\neg \text{Prerequisite}(\text{AI}, p) \vee \neg \text{Completed}(x, p) \vee \text{CanEnroll}(x, \text{AI})$

Resolve with $\text{Prerequisite}(\text{AI}, \text{DataStructures})$ [$\theta = \{p/\text{DataStructures}\}$]:

$\neg \text{Completed}(x, \text{DataStructures}) \vee \text{CanEnroll}(x, \text{AI})$

Resolve with $\text{Completed}(\text{Joanna}, \text{DataStructures})$ [$\theta = \{x/\text{Joanna}\}$]:

$\text{CanEnroll}(\text{Joanna}, \text{AI})$

Resolve with the negated goal $\neg \text{CanEnroll}(\text{Joanna}, \text{AI})$:

■ (the empty clause)

Deriving the empty clause (■) is a contradiction, which proves that the negated goal cannot hold together with the knowledge base; hence $\text{CanEnroll}(\text{Joanna}, \text{AI})$ is proved true by refutation.

9. FOL-based Approach vs. Propositional-Logic Implementation

In propositional logic, each fact and rule must be written out separately for every student and every course, since propositional logic has no variables or quantifiers. For example, R1 above would have to be duplicated as a separate rule for every (student, course, prerequisite) combination:

P1: $\text{Course_AI} \wedge \text{Prereq_AI_DS} \wedge \text{Completed_Joanna_DS} \rightarrow \text{CanEnroll_Joanna_AI}$

P2: $\text{Course_AI} \wedge \text{Prereq_AI_DS} \wedge \text{Completed_Ravi_DS} \rightarrow \text{CanEnroll_Ravi_AI}$

...and a further separate propositional rule would be required for every additional student or course added to the university.

Aspect	Propositional Logic	First-Order Logic
Representation	One ground statement per case	One general rule with variables covers all cases
Scalability	Rule count grows with number of students/courses	Knowledge base size stays constant as data grows
Quantification	Not supported (no $\forall / \exists$)	Supports universal and existential quantifiers
Inference	Simple truth-table / resolution on fixed atoms	Unification-based resolution over variables and terms
Reuse	Cannot generalize a rule to a new individual	Same rule (e.g. R1) automatically applies to any new student
Expressiveness	Limited to atomic true/false facts	Can express relations, objects, and properties over objects

In summary, the FOL-based advisory agent uses variables (x, c, p, g, i), quantifiers ($\forall$), predicates over objects, and unification to write a small, compact set of 6 general rules that correctly reason about any number of students and courses, whereas an equivalent propositional-logic system would need a separate hard-coded rule for every student-course combination, making it far less compact, harder to maintain, and unable to generalize to new students or courses without adding new propositions by hand.